VeriFlow Guarded Preservation and Sparse Replay of Row Pipelines Across Schema Migration

Abstract

A schema migration can discard source values while leaving the behavior of a particular data pipeline unchanged. Conversely, a small numeric conversion can change a filter decision even when the pipeline still runs. We present VeriFlow, a compiler and migration prototype for deterministic, typed, row-wise CSV pipelines with derivation, validation, filtering, and projection. Given an explicit one-step schema mapping, VeriFlow identifies observed old fields and derives sufficient preservation rules or executable guards. It compiles bindings from new-schema cells to the existing pipeline VM, then chooses guard admission, selective old-cell retention, or rejection for each migrated row. A persisted bundle records row IDs, field-level guard certificates, retained cells, and replayed typed observations. On an actual 2021 Malaysian Ministry of Health ICU schema change, 7,315 paired rows show a removed field with nonzero values on 4,060 rows. A pipeline that ignores the removed field is preserved with no retention; a pipeline that selects it is replayed by retaining one cell per row instead of five under full-row retention. All processed observations and all compared post-change rows matched. A separate ten-case synthetic pilot over 5,926 distinct public-data rows admitted 10,379 case-rows and recovered 7,549, with no observed replay mismatch. The guarantee applies to well-typed legacy rows at migration time, not arbitrary future new-schema rows.

Keywords: schema evolution; data pipelines; conditional preservation; guarded migration; provenance; executable replay.

1 Introduction

Schema-change labels describe record reconstruction, but a pipeline only observes some source values and decisions. Suppose `score: decimal` becomes `score: int` by truncation toward zero. A row with score `40.7` changes the truth of `score > 40`, but not `score >= 40`. If the pipeline only filters on `score >= 40` and selects an unchanged `name`, it can preserve its typed observation even though the exact score is lost. A second change may drop an old field that the pipeline never reads, leaving behavior unchanged. If the pipeline selects that field, continued execution instead requires retained information.

VeriFlow asks three questions. **RQ1:** Can a restricted compiler identify when a lossy mapping preserves a specific row pipeline, and state a checkable condition for uncertain values? **RQ2:** Can migration turn that analysis into an executable new-schema input plan that admits, retains, or rejects each row and replays the result from disk? **RQ3:** What admission, retention, and byte costs arise on synthetic stress cases and one documented real schema change?

The contribution is a scoped executable preservation contract. VeriFlow combines dependency-sensitive proof rules, migration-time guard admission, per-field certificates, sparse retained cells, and persisted replay. It also reports a collision when two distinct old rows have the same migrated row but different old observations. The system uses explicit mappings; it does not infer a migration, rewrite a textual new `.vfl` program, or prove behavior for new rows that did not originate from checked old rows.

2 Model and preservation contract

A legacy schema defines typed fields. A VeriFlow pipeline orders `let`, `require`, and `keep` operations before `select`. Its VM evaluates exact rational numeric values, text values, and Boolean expressions. For a well-typed old row `s`, `Obs(P,s)` is one of: emitted selected typed values in order; filtered; rejected with its validation message; or expression error with its error class. An executed `let` matters even when unselected because its expression can fail. This observation model excludes the original lexical spelling of CSV cells and malformed-input handling.

Let `F` be a defined explicit forward mapping from old to new rows. Let `E` be the compiled evolved execution plan: new-schema field bindings followed by the legacy VM instructions. For each field that an executed operation or selected output can observe, the compiler supplies either exact reconstruction, a sufficient guard checked on `s`, or a retained old cell. A row accepted by the migration command satisfies:

```
Obs(P, s) = Obs(E, F(s), retained(s), certified(s))
```

Here `certified(s)` names guarded bindings whose guards passed on the old row; `retained(s)` supplies old values for required fields whose guard failed or whose source field was dropped. A field-level certificate matters when one guard passes and another fails on the same row. Certificates are migration assertions stored in the bundle, not proofs that a later arbitrary new-schema row satisfies a guard. The bundle uses unkeyed digests to detect accidental inconsistency; it is not an authenticated record against an attacker who can rewrite the whole bundle.

VeriFlow reports preserved when the sufficient condition is universal on its stated domain, conditional when a guard is required, breaking when a replayed old-row collision shows a new-row-only deterministic program cannot preserve both observations, and unknown when no supported rule or collision decides the case. Absence of a bounded witness never establishes preservation.

2 1 Sufficient rules

Rule	Scope and argument
Exact reconstruction	Same-type copy/rename and int to decimal reconstruct the same typed source value.
Unused source	A dropped or changed field outside the transitive dependencies of executed operations and selected outputs cannot affect Obs on a well-typed row.
Positive integer threshold	For finite decimal x and integer k >= 1, truncation toward zero preserves x >= k and x < k; pointwise preservation extends through supported Boolean composition.
Threshold-gap guard	For one affected atomic x > k or x <= k, the truth value changes exactly when k < x < k+1. The guard admits values outside this interval. Conjoined guards are sufficient, not generally optimal, for compound expressions.
Integral fallback	If an old decimal is integral, decimal-to-integer conversion reconstructs its exact typed value. This covers selected values and operations outside the narrower threshold rule.
Text split/merge guards	The guard checks the actual round trip of the defined split or merge operator. For split without a delimiter occurrence, the empty second part causes reverse joining to append a delimiter, so that case needs retention.

The exact-value argument follows induction over ordered VM operations: unchanged inputs give unchanged derived values, short-circuit choices, validation outcomes, filtering decisions, and selected outputs. The threshold rule follows from the integer intervals on each side of positive k. Division-sensitive derivations fall back to exact integral reconstruction because truncation can change a nonzero denominator into zero. An observed dropped field without retention remains unknown or breaking; the system does not turn a missing value into a valid old value.

3 System design

The existing compiler lexes and parses a .vfl pipeline, checks types, builds intermediate operations, and runs a VM. A separate evolution parser and compiler classify explicit copy, rename, conversion, add, drop, split, and merge mappings. Impact analysis follows source-field dependencies through derived expressions and executed conditions. compile-evolved generates an EvolvedProgram with new-schema input fields, bindings for observed old fields, guard/recovery requirements, and the existing VM bytecode. The bindings fill VM register names; this is a compiled new-schema input plan rather than a textual source-to-source rewrite.

migrate-run reads each old row once, validates its cells, evaluates guards before lossy conversion, and migrates it. In sparse mode it retains a dropped observed field or a guarded field whose guard fails. For other guarded fields on the same recovered row, it saves field-level certificates. Guard-only mode rejects rows needing retention; full-row mode retains every old source field. Each accepted row gets a UUID within the bundle. The bundle stores migrated.csv, ledger.jsonl, sidecar.jsonl, program.json, manifest.json, replay.jsonl, rejects, and a summary. replay-migration loads the persisted plan without original source or mapping files, verifies paired row IDs and digests, and executes from new cells plus retained old cells. A mismatch with the original typed observation aborts bundle publication.

The compiler review uncovered a mixed-guard defect in an earlier version: a recovered row was treated as if all its guards had failed, so it could reject a valid row when only one field needed retention. The implemented field-level certificate fixes this case. A regression uses a = 41.7, b = 40.7, two strict-threshold guards, and a decimal-to-integer conversion; it retains only b and replays the row. A second regression combines a passing guard with an observed dropped field.

4 Evaluation design

We evaluate three existing strategies: **sparse** (admit or selectively retain), **guard-only** (admit or reject), and **full-row** (retain all declared old cells). A schema-level loss label and a dependency-only checker are diagnostic internal ablations, not competing research systems. We report counts, retained cells, sidecar bytes, total bundle bytes, three-run median end-to-end runtime, and typed-observation mismatches. The timings include analysis, migration, file writing, and disk replay. A guard-only run that rejects rows does less work, so its runtime is not an isolated guard overhead.

4 1 Versioned real schema change

The Malaysian Ministry of Health public repository versions the ICU CSV. The 22 July 2021 commit removes vent_sari and renames four beds_icu* fields to bed_icu*. We use its immediate parent and the changed commit as the paired versions. The included projections retain date, state, the relevant bed count, vent_covid, and the old vent_sari. A verifier pins both original Git blobs and both projected SHA-256 hashes, checks 7,315 unique (date,state) pairs, and checks that every actual post-change projected row equals the rename-and-drop mapping of its old counterpart. The removed value is nonzero on 4,060 old rows. This is observed schema and data history, while the two tested pipeline definitions are ours rather than reported Ministry applications.

The **preserved** pipeline filters on vent_covid > 0 and selects date, state, and the renamed bed count. It never reads the dropped field, so it should require no retention. The **recovered** pipeline applies the same filter and selects date, state, and dropped vent_sari; it should require one retained old cell for every source row. For both, we check each accepted migrated row against the actual official post-change projection and compare replayed typed observations with old execution. These two workloads reuse the same 7,315 source rows and are never counted as independent records.

4 2 Synthetic stress and controls

Ten explicit injected changes over UCI Abalone, Iris, and Red Wine Quality reuse 5,926 distinct source rows across 17,928 case-rows. They cover safe and strict decimal-to-integer thresholds, selected changed values, and a guarded text split. Eight predeclared negative controls probe validation, derived expressions, expression errors, observed drops, unsupported conversion, and Boolean combinations. Four CSV parity checks compare outputs and rejects on guard-admitted rows. An observed-drop collision control and a two-row NYC311 field-addition reenactment are reported separately. NYC311 uses experimental defaults and is not treated as an official migrated snapshot.

All final integrated runs used Python 3.12.14 on Windows. The full test suite passed 103 tests. The integrated study runs each strategy three times per case and records medians. The raw results and exact commands are in evaluation/.

5 Results

5 1 Real paired history

Pipeline and strategy	Admitted	Recovered	Rejected	Old cells retained
Preserved / sparse	7,315	0	0	0
Preserved / guard-only	7,315	0	0	0
Preserved / full-row	0	7,315	0	36,575
Recovered / sparse	0	7,315	0	7,315
Recovered / guard-only	0	0	7,315	0
Recovered / full-row	0	7,315	0	36,575

Pipeline and strategy	Sidecar bytes	Bundle bytes	Actual after rows checked	Typed mismatches
Preserved / sparse	0	2,682,668	7,315	0
Preserved / guard-only	0	2,682,676	7,315	0
Preserved / full-row	1,728,544	4,872,071	7,315	0
Recovered / sparse	1,141,397	4,281,009	7,315	0
Recovered / guard-only	0	861,956	0	not applicable
Recovered / full-row	1,728,544	4,868,161	7,315	0

Every compared new row also matched the actual post-change projection. For the recovered pipeline, sparse mode retains 80% fewer old cells and 34.0% fewer sidecar bytes than full-row retention. It reduces total bundle bytes by 12.1% in this workload. The guard-only recovered case rejects every row, so its small bundle and zero replay mismatches are not preservation evidence for those rows. On the preserved pipeline, sparse mode requires no sidecar despite a genuinely dropped source field.

The three-run median times for the real preserved case were 2,054 ms sparse and 2,717 ms full-row. For the real recovered case they were 2,592 ms sparse and 2,741 ms full-row. These are descriptive single-host measurements; no statistical significance or general speedup is claimed.

5 2 Injected pilot

Strategy	Admitted	Recovered	Rejected	Old cells retained	Sidecar bytes	Typed mismatches
Sparse	10,379	7,549	0	7,549	1,221,174	0
Guard-only	10,379	0	7,549	0	0	0
Full-row	0	17,928	0	53,784	3,452,534	0

The pilot's 17,928 case-rows reuse 5,926 distinct rows; they are not 17,928 independent observations. Three record-lossy but pipeline-preserving numeric cases pass without guard restrictions. Strict-threshold guards admit 2,782/4,177 Abalone, 99/150 Iris, and 1,214/1,599 Red Wine rows. The observed-drop two-row control shows different old selected values after both old rows collide into the same new row; sparse retention restores both. These checks support the scoped implementation, while the synthetic changes cannot establish prevalence in real histories.

6 Related work

Awiti, Wrembel, and Zimányi (2025) repair ETL processes affected by source schema changes using extended relational algebra and operator-specific repair rules. VeriFlow addresses the narrower question of preserving typed row observations under an explicit mapping, including value-dependent guards, selective retention, and paired replay. It does not claim general ETL repair as new. Kramer, Restat, and Störl (2025) analyze operator robustness and propose directions for self-adaptive cleaning pipelines; our executable pilot studies one limited operator fragment rather than implementing their broader vision.

Auge and Heuer (2022) use provenance to enhance inversion of schema evolution, so storing auxiliary source information is established prior art. VeriFlow's contribution is the binding of such retained values to a pipeline-specific preservation decision and replay bundle. Vassiliadis et al. (2023) study schema and source histories across 195 projects, motivating explicit real-history evaluation. EvoSchema (2025) benchmarks schema-evolution robustness for text-to-SQL systems; its task and metrics are different, but its diversity of schema perturbations underscores the narrowness of our one-change real study. We do not claim conditional equivalence, selected-information preservation, or schema rewriting as concepts invented here.

7 Threats to validity and limits

The theorem concerns finite, well-typed legacy rows with defined mappings and migration-time access to their original values. The compiled input plan fills legacy VM registers; it is not a standalone textual rewritten pipeline. New-only future rows have no old-row guard certificate, so the result does not certify their behavior. The language excludes joins, aggregation, arbitrary UDFs, and multi-step migrations. Guard conjunction can conservatively retain rows whose full compound expression would actually remain unchanged. Sparse retention is sufficient, not globally minimal.

The Malaysian case is one real drop-and-rename event with two experimental pipelines, not a sample of organizational production pipelines. Its official paired data give stronger external evidence than an injected mutation, but one event cannot establish generality. The other ten cases use injected changes over reused rows. The reported zero mismatches are observed finite-study results, not an empirical proof of the universal rules; the rule arguments carry the scoped claim. Runtime medians use three repetitions on one Windows machine and include different amounts of work by strategy. Byte totals depend on JSONL and CSV serialization and UUID lengths. The bundle's unkeyed hashes detect accidental corruption but do not provide adversarial authentication or stable source identity across migrations.

8 Reproducibility and conclusion

From the repository root with Python 3.12 or later:

```
python -m evaluation.real_history.prepare_moh_icu
python -m unittest discover -s tests -q
python -m evaluation.migration_study
python -m evaluation.freeze_manifest
```

The first command verifies included projections without downloading data; `--fetch` regenerates them from pinned official commits when network access is available. The study writes `evaluation/migration-results.json`; the manifest hashes code, case definitions, data, and results. `compile-evolved`, `migrate-run`, and `replay-migration` provide a direct CLI demonstration, as specified in the method note.

VeriFlow makes a limited but executable claim: a lossy source schema change need not change a specific pipeline's observations, and when it does remove information the system can retain observed cells and replay a migrated bundle. The real paired ICU history demonstrates both paths on the same documented change. Broader schema classes, arbitrary future-row guarantees, and independent multi-organization evaluation remain open.

References

1. Awiti, J., Wrembel, R., and Zimányi, E. (2025). Repairing ETL Processes using Extended Relational Algebra. Foundations of Computing and Decision Sciences, 50(2), 157–190.
2. Kramer, K. M., Restat, V., and Störl, U. (2025). Evolving Gracefully: Building Robust and Self-Adaptive Data Cleaning Pipelines for Schema Evolution and Uncertainty. QDB at VLDB 2025.
3. Auge, T., and Heuer, A. (2022). Enhanced Inversion of Schema Evolution with Provenance. arXiv:2211.13810.
4. Vassiliadis, P., Shehaj, F., Kalampokis, G., and Zarras, A. V. (2023). Joint Source and Schema Evolution: Insights from a Study of 195 FOSS Projects. EDBT 2023.
5. Zhang, T., et al. (2025). EvoSchema: Towards Text-to-SQL Robustness Against Schema Evolution. PVLDB, 18(10), 3655–3668.
6. Ministry of Health Malaysia. (2021). Official COVID-19 public dataset, ICU schema-change commit.

Appendix A Artifact details

The actual historical comparison uses the parent commit `3b56bc326306a013d1fb26eb4f33084f432b4b4c` and changed commit `4e80229047d796f14c7aac82910a3377277b699f`. The corresponding official `epidemic/icu.csv` Git blob hashes are `f8bd4e11746df2109807e7a1f0261982b4605564` and `bc45a1fa903e0b5ec32ccb7a6ca0b37436e10a37`. The included projected CSV SHA-256 values are `b8a0d35cc73b45f6b9367f67472ee2fbc01559993185ce55694317d2d0f651f6` before and `2cd79c6da5e714153456ee743d8f6d2925c2b108b4997ffa0690215911036be6` after. The projection script verifies the source Git blobs when run with `--fetch` and checks all 7,315 date/state pairs in its ordinary offline mode.

The real preserved pipeline's three-run median times were 2,054 ms sparse, 2,021 ms guard-only, and 2,717 ms full-row. The real recovered pipeline's medians were 2,592 ms sparse, 238 ms guard-only, and 2,741 ms full-row. The 238 ms guard-only run rejected all 7,315 rows, so that timing is not a comparable throughput result. The counted bundle bytes include the new-schema CSV, ledger, persisted program, sidecar, replay, rejects, manifest, and summary. Source and case files are outside that bundle measurement. A repeated study run generates new UUIDs, but the measured file lengths and counts are checked for agreement across repetitions.

The direct regression for mixed guards is in `tests/test_migration.py`. It checks a recovered row with one certified field and one retained field, then removes the certificate and verifies that source-free replay refuses the incomplete bundle. The full suite's 103 passing tests include compiler, optimizer, evolution, impact, recovery, migration, runtime, and CLI checks. This is an implementation check, separate from the formal sufficiency arguments and finite dataset comparisons.